TRABAJO PRACTICO ESPECIAL

Sistemas Embebidos

Descripción breve

En este informe se hará un análisis detallado del trabajo practico especial desarrollado en la materia Sistemas Embebidos

Burdisso Jorge Tobias
tobiasburdisso@gmail.com

Introducción

En este informe se detallará el desarrollo del trabajo practico especial de la materia. El mismo consiste en utilizar el STM32F103C8 juntos con otros periféricos para realizar un reproductor de audio de archivos WAV. El sistema tiene que ser capaz de:

- Leer los archivos WAV (mono/8-bits/8kHz) desde una tarjeta SD.
- Seleccionar el archivo a reproducir con un joystick
- Aplicar filtros digitales de dos bandas tanto a las frecuencias bajas como altas.
- Calcular la transformada de Fourier de 64 puntos para visualizar las frecuencias en un pantalla OLED utilizando un efecto peak-hold.
- Generar audio via PWM
- Filtrar y amplificar dicha señal para su posterior reproducción a través de un parlante

Alcances y limitaciones

Implementamos: lectura de archivos WAV desde la SD utilizando FATFS , un procesamiento de muestras con dos filtros digitales biquad en cascada (HP + LP), cálculo de FFT de 64 puntos, salida por PWM (TIM3) sincronizada con un timer a 8 kHz (TIM2) y una interfaz de usuario en OLED con selección de archivo mediante un joystick (ADC) y botón.

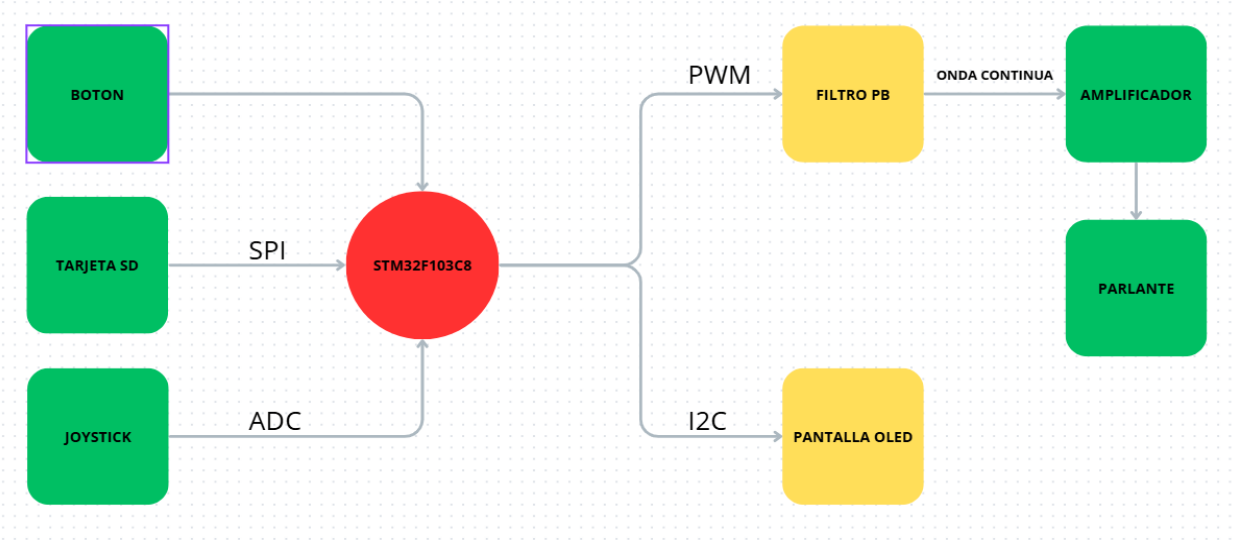
Limitaciones reales del desarrollo: no logre mostrar en la OLED el gráfico de la FFT en tiempo real. El cálculo de la FFT y la actualización de la pantalla introducían latencias que no pudieron sincronizarse con la rutina de salida (PWM/TIM2) sin provocar interrupciones perceptibles en la reproducción. Por ello documento la aproximación implementada, las causas detectadas y las posibles soluciones futuras

Desarrollo

En este apartado hare el desarrollo completo del sistema mostrando en primer lugar la implementación física, es decir, los periféricos utilizados y como estos se conectan y comunican con el sistema. Después mostrare el apartado de software, donde explicare las diferentes tareas, como se comunicación entre si, sus prioridades, su tamaño en memoria etc.

Hardware y comunicación

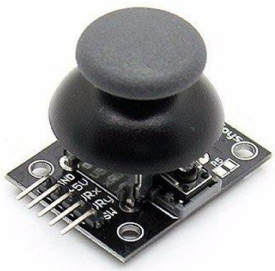
El sistema se basa en el microcontrolador STM32F103C8 como unidad de procesamiento, se conectará con diferentes periféricos leyendo información de algunos y enviando a otros. El esquema general de conexión y las comunicaciones utilizadas son las siguientes:



Modulo lector de Tarjeta SD: se utiliza una comunicación SPI a través de los pines se conecta con el microcontrolador a través de los pines **PA7** (MOSI) **PA6** (MISO) **PA5** (SCK) y **PB1** (CS) y se alimenta con **5V** o **3.3V**, la configuración fue la siguiente:

Basic Parameters	
Frame Format	Motorola
Data Size	8 Bits
First Bit	MSB First
Clock Parameters	
Prescaler (for Baud Rate)	256
Baud Rate	250.0 KBits/s
Clock Polarity (CPOL)	Low
Clock Phase (CPHA)	1 Edge

Joystick: el microcontrolador lee los valores a través del conversor analógico-digital (pin **PA0**) modo de conversión única disparado por software, se alimenta con **5V**. Es utilizado para la selección del archivo WAV a través de la pantalla OLED.



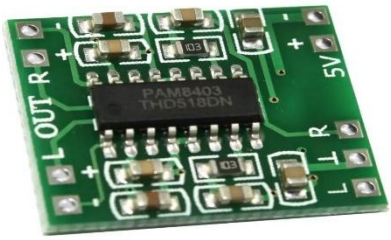
ADCs_Common_Settings	
Mode	Independent mode
ADC_Settings	
Data Alignment	Right alignment
Scan Conversion Mode	Disabled
Continuous Conversion Mode	Disabled
Discontinuous Conversion Mode	Disabled
ADC_Regular_ConversionMode	
Enable Regular Conversions	Enable
Number Of Conversion	1
External Trigger Conversion Source	Regular Conversion launched by software
Rank	1

Botón: utilizado para seleccionar el archivo a reproducir, genera una interrupción a través del pin **PA15**, tiene habilitado una resistencia de pull-up interna.

Filtro PB: se trata de un filtro pasa bajo conformado por una resistencia de 220 ohm y dos capacitores en serie de 100nF. Con estos logramos una frecuencia de corte de:

$$\boldsymbol{fc = \frac{1}{2\pi RC} = \frac{1}{2\pi * 220\Omega * 50nF} = 14.47\ kHz}$$

Amplificador: el PAM8403 es un amplificador de audio de clase D de 3W. Se pueden aplicar dos canales, pero como nuestra salida es mono utilizamos solo uno. Se alimenta con un rango de **2.2V a 5V.**



Pantalla OLED: se trata del la pantalla SSD1306, el microntrolador se comunica a traves de I2C (SCL y SDA) pines **PB6** y **PB7.** Se muetran las canciones a reproducir y los frecuencias de ecualizacion. Se alimenta con 3.3V

Master Features

I2C Speed Mode	Fast Mode
I2C Clock Speed (Hz)	400000
Fast Mode Duty Cycle	Duty cycle Tlow/Thigh = 2

Software

En esta sección explico con detalle cómo diseñé e implementé el software del reproductor. Mantengo la reproducción como prioridad absoluta y organizo todo lo demás para que no interfiera con la salida de audio en tiempo real.

El sistema corre sobre FreeRTOS y lo dividí en 5 tareas con responsabilidades bien definidas. Pero antes de explicar cada tarea hare una descripción del código en general librerías, funciones que utilizo, configuración del sistema etc.

Librerias

Las librerías de interés son **“fattfs.h”, “ssd1306.h”** y “**fft.h”** donde la primera la utilizo para manipular la lectura de los archivos WAV la segunda para dibujar/escribir en la pantalla OLED y la ultima para realizar la transformada de Fourier de las muestras.

```
#include "main.h"
#include "cmsis_os.h"
#include "fatfs.h"

/* Private includes ---------
/* USER CODE BEGIN Includes

#include "FreeRTOSConfig.h"
#include "FreeRTOS.h"
#include "ssd1306.h"
#include "fft.h"
```

Funciones

- **Init_Display():** Como el nombre lo indica inicializa el display, escribiendo “**Reproductor WAV**” en la pantalla.
- **ScanWavFiles():** escanea la tarjeta SD y obtiene los nombre de los archivos terminados en “.wav”. Almacena los nombres en un arreglo de dos dimensiones wavFiles[MAX_WAV_FILES][MAX_NAME_LEN].
- **OLED_DisplayString(char *str, uint8_t x, uint8_t y):** escribe una cadena en la pantalla en la posicione deseada. Escribe únicamente si está libre el mutex.
- **iniciar_reproduccion_pwm() – detener_reproduccion_pwm():** como el nombre lo indica inicio y detengo la reproducción del PWM, dejo de disparar la interrupción para obtener muestras e ir cambiando el duty cycle y detengo le TIM3 responsable de generar la señal PWM.
- **filtro_iir(float x, Biquad_t *f):** función para aplicar el filtro digital, toma como parámetro la muestra y un puntero a una estructura biquad la cual defini yo para almacenar los valores de los coeficientes del filtro. Estos coeficientes los obtuve de biquad calcualtor.

- **fast_sqrtf(float number):** como *sqrt* tiene un costo alto de memoria utilizo el método de Newton-Raphson para encontrar la raíz cuadrada.

```
void Init_Display(void);
void ScanWavFiles(void);
void OLED_DisplayString(char *str, uint8_t x, uint8_t y);
void iniciar_reproduccion_pwm(void);
void detener_reproduccion_pwm(void);
float filtro_iir(float x, Biquad_t *f);
float fast_sqrtf(float number);
```

Timers

Utilizo el timer 2 que sirve como base de muestreo para la reproducción de audio. Está configurado para generar una interrupción cada 125 µs (8 kHz), que coincide con la frecuencia de muestreo de los archivos WAV utilizados. En cada interrupción, leo la siguiente muestra del buffer PWM y actualizo el valor de comparación (CCR1) del canal PWM de TIM3. Cuando se alcanza el final de un bloque, TIM2 libera el semáforo **sem2** para que **TASK_EcuFilter** cargue el siguiente bloque sin sobrescribir datos aún no reproducidos.

El timer 3 se usa en **modo PWM** para generar la señal de audio. Está configurado con una frecuencia de **100 kHz** y un valor de recarga (ARR) que produce una resolución de **640 pasos** para el duty cycle. Cada muestra procesada se traduce en un valor de CCR1 (entre 0 y 640) que modula la amplitud de la señal PWM. Elegí una frecuencia de 100 kHz porque al ser mucho mayor que la banda de audio, el contenido de alta frecuencia queda bien separado del audio, lo que facilita su filtrado con un simple pasa-bajo y reduce el ruido audible.

Colas

A continuación, se mencionan todas las colas que hay en el programa, que tareas comunican y cuál es su tamaño.

- **Queue_SongIndex**: transmite el índice de la pista elegida desde la tarea de interfaz (**TASK_UI**) hacia la tarea de lectura de SD (**TASK_LecturaSD**). Solo requiere un byte y una profundidad de un elemento, ya que se envía un único valor cada vez que el usuario confirma la selección.
- **Queue_EcuFilter**: envía bloques de datos leídos desde la SD hacia la tarea de procesamiento (**TASK_EcuFilter**). Cada bloque contiene 256 bytes del archivo WAV. Es la cola más crítica del sistema, ya que asegura que el procesamiento reciba datos de manera continua sin depender directamente del tiempo de lectura de la tarjeta.
- **Queue_FFT**: recibe ventanas de 64 muestras generadas por **TASK_EcuFilter** y las envía a la tarea de análisis (**TASK_FFT**). Su función es desacoplar la generación de datos de

la FFT del cálculo espectral, que es más costoso. En caso de saturación, las ventanas pueden descartarse sin afectar la reproducción.

Queues

Queue Name	Queue Size	Item Size	Allocation	Buffer Name	Control Block Name
Queue_SongIndex	1	uint8_t	Dynamic	NULL	NULL
Queue_EcuFilter	1	256	Dynamic	NULL	NULL
Queue_FFT	1	256	Dynamic	NULL	NULL

Mutex y semáforos

En el este sistema utilizo dos semáforos binarios y un mutex. El primer semáforo **sem1** sirve para coordinar la selección de canciones desde la tarea UI como así también cambiar entre el modo de funcionamiento **UI_MODE** y **FFT_MODE.** Este bloque la **defaultTask** liberándola únicamente cuando oprimo el botón para seleccionar la canción o para detenerla.

El segundo semáforo **sem2** se utiliza para coordinar el llenado del buffer PWM entre la ISR de muestreo y la tarea **TASK_EcuFilter**. La ISR de TIM2 libera este semáforo cuando termina de reproducir un bloque completo de audio, permitiendo que **TASK_EcuFilter** escriba un nuevo bloque. De esta forma se evita que el procesamiento sobrescriba datos que aún no fueron enviados a la salida.

Luego por último el **mutex1** protege el acceso a la pantalla OLED, dado que la comunicación por I2C no es atómica y varias tareas pueden intentar escribir en la pantalla, el mutex garantiza que solo una lo haga a la vez, evitando corrupción gráfica.

Binary Semaphores

Semaphore Name	Allocation	Control Block Name	Initial State
sem1	Dynamic	NULL	Depleted
sem2	Dynamic	NULL	Available

Mutexes

Mutex Name	Allocation	Control Block Name	State
mutex1	Dynamic	NULL	Depleted

Tareas

Descripción de las tareas su función y prioridad.

- **TASK_UI**: gestiona la interfaz de usuario. Lee el joystick para seleccionar la canción, muestra la información en la pantalla OLED y envía el índice elegido a través de **Queue_SongIndex**. Es una tarea de baja prioridad ya que su ejecución no es crítica para el audio.
- **TASK_LecturaSD**: se encarga de abrir el archivo WAV seleccionado, saltar la cabecera y leer bloques de 256 bytes desde la tarjeta SD utilizando FATFS. Cada bloque leído se envía a **Queue_EcuFilter** de manera bloqueante. Esta tarea tiene prioridad media-alta para asegurar un flujo constante de datos hacia el procesamiento.

- **TASK_EcuFilter:** recibe los bloques desde **Queue_EcuFilter**, y a su vez espera que se liebre el **sem2**, esto indica que se terminó de leer el buffer pwm indicando asi que ya puede volver leer. Luego convierte las muestras a un rango centrado entre -1 y 1, aplica dos filtros biquad en cascada, uno pasa altos de 200Hz y otro pasa bajos de 3600Hz para luego almacenar el resultado en el buffer PWM. Además, extrae una de cada cuatro muestras para formar ventanas de 64 puntos que envía a **Queue_FFT** de manera no bloqueante. Es la tarea de mayor prioridad, ya que alimenta directamente la salida de audio.
- **TASK_FFT**: toma las ventanas de **Queue_FFT**, ejecuta la FFT de 64 puntos, calcula magnitudes y prepara los datos para la visualización en la OLED. En esta versión, el dibujo quedó deshabilitado para no interferir con la reproducción. Su prioridad es media, ya que su ejecución no debe interrumpir las tareas críticas de audio.
- **DefaultTask**: al inicio del sistema esta tarea permanece bloqueada en la espera de sem1, sin consumir CPU. Cuando el usuario selecciona una canción en la interfaz y confirma con el botón, la rutina de interrupción asociada a ese botón libera sem1. Esto provoca que **DefaultTask** se ejecute, cambiando el modo de operación global de **UI_MODE** a FFT_MODE. Al mismo tiempo, la tarea envía el índice de la pista seleccionada a **Queue_SongIndex** para que la tarea **TASK_LecturaSD** abra el archivo y comience la reproducción. Tiene la prioridad mas alta esta tarea que indica el comienzo y fin de la reproducción.

Tasks

Task Name	Priority	Stack Size (Words)	Entry Function	P...	Code Generati...	Allocat...	Buffer Name	Contr...
defaultTask	osPriorityHigh4	128	StartDefaultTask	N...	Default	Dyna...	NULL	NULL
TASK_UI	osPriorityAboveNormal4	200	xTASK_UI	N...	Default	Dyna...	NULL	NULL
TASK_LecturaSD	osPriorityAboveNormal6	512	xTASK_LecturaSD	N...	Default	Dyna...	NULL	NULL
TASK_EcuFilter	osPriorityAboveNormal7	384	xTASK_EcuFilter	N...	Default	Dyna...	NULL	NULL
TASK_FFT	osPriorityAboveNormal5	450	xTASK_FFT	N...	Default	Dyna...	NULL	NULL

Uso de memoria y analizador lógico

Podemos ver el análisis estático de la stack cuando depuramos. Donde la tarea **Lectura SD** es la que más consume debido al buffer para el envió de muestras y a las variables de la librería FATFS.

Function	Depth	Max cost	Local cost	Type
ADC1_2_IRQHandler	?	?	0	
xTASK_LecturaSD	9	1704	840	STATIC
Reset_Handler	13	1596	0	
xTASK_FFT	5	1008	832	STATIC
xTASK_EcuFilter	5	760	560	STATIC
xTASK_UI	8	468	24	STATIC

Luego si vemos la pantalla del IOC podemos ver en las configuraciones y el HEAP total utilizado.

HEAP STILL AVAILABLE	1151 Bytes
TOTAL HEAP USED	8349 Bytes
Total amount for tasks	7296 Bytes
Total amount for queues	789 Bytes
Total amount for timers	0 Bytes
Total amount for mutexes and semaphores	264 Bytes
Total amount for events	0 Bytes

Por último, podemos ver la captura de pantalla del analizador lógico donde se ven las tareas **EcuFilter**, **Lectura SD** y **FFT** corriendo, donde se puede apreciar como **Ecufilter** corre cada 32ms que es la cantidad de bytes que tiene el buffer por la velocidad de lectura 256 bytes * 125us = 32ms. Podemos ver que el tiempo que no corre **EcuFilter** se reparte entre las otras dos tareas. Esto me imposibilitaba poder escribir en la pantalla ya que tardaba mucho tiempo, provocando que la interfaz quede congelada y si le daba mas prioridad que el audio hacia que se escuche mal.

Conclusión

Si bien el desarrollo del sistema no fue del todo exitoso los aspectos mas relevantes del trabajo a destacar son.

Logros principales:

- Se implementó con éxito la lectura de archivos WAV desde una tarjeta SD utilizando la librería FATFS.
- Se logró reproducir audio mediante PWM, sincronizado con un timer a 8 kHz, y se aplicaron filtros digitales biquad en cascada (pasa-altos y pasa-bajos) para mejorar la calidad del sonido.

- La interfaz de usuario, controlada por un joystick y un botón, permitió la selección de archivos y mostró información en una pantalla OLED, aunque con limitaciones en la visualización en tiempo real.

Desafíos y limitaciones:

- La principal dificultad fue la sincronización entre el cálculo de la FFT y la actualización de la pantalla OLED, lo que generó latencias que afectaron la visualización en tiempo real. Esto se debió a los altos requerimientos computacionales de la FFT y las limitaciones del hardware.
- La priorización de tareas en FreeRTOS fue crítica para mantener una reproducción de audio sin interrupciones, pero esto relegó funciones secundarias como la visualización de la FFT.

Posibles mejoras:

- Explorar el uso de DMA (Acceso Directo a Memoria) para liberar al microcontrolador de tareas de transferencia de datos y mejorar el rendimiento general.
- Optimizar el calculo de la FFT y priorizar mejor las tareas y sincronización entre estas.

En conclusión, pude implementar un reproductor de audio funcional basado en el STM32F103C8, cumpliendo con los objetivos principales: lectura de archivos WAV desde una tarjeta SD, reproducción mediante PWM, aplicación de filtros digitales y una interfaz de usuario básica. Sin embargo, surgieron limitaciones técnicas, especialmente en la visualización en tiempo real de la FFT debido a restricciones de procesamiento y sincronización.